Forge: A Robust Multi-tab Website Fingerprinting Attack Via Blind Source Separation

Yitan Huang
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
huangyitan@iie.ac.cn

Wei Qiao
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
qiaowei@iie.ac.cn

Ding Wang
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
wangding2023@iie.ac.cn

Meng Shen
School of Cyberspace Science and Technology, Beijing Institute of Technology
Beijing, China
shenmeng@bit.edu.cn

Di Zhao
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
zhaodi@iie.ac.cn

Linxu Li
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
lilinxu@iie.ac.cn

Susu Cui
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
cuisusu@iie.ac.cn

Bo Jiang
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
jiangbo@iie.ac.cn

Zhigang Lu
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
luzhigang@iie.ac.cn

Baoxu Liu
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
liubaoxu@iie.ac.cn

Abstract

While Tor's strong anonymity shields users' privacy, it also enables malicious activities, motivating attacks that bypass its protections. Website Fingerprinting (WF) has emerged as a primary threat in this domain. However, existing WF methods struggle with realistic multi-tab browsing scenarios, often relying on prior knowledge of the number of open tabs and lacking robustness against network noise and defenses.

To address these challenges, we propose Forge, a robust WF attack framework inspired by the classic cocktail party problem. Specifically, Forge reframes multi-tab WF as a task of Blind Source Separation(BSS), decomposing mixed traffic into individual signals without requiring a predefined number of concurrent tabs. A robust website identifier then classifies separated components using a dual-domain attention mechanism across time and frequency, allowing Forge to effectively resist WF defenses and network noise. We evaluate our model on a comprehensive collection of datasets covering open-world, defense-enabled, and dynamic scenarios. The results demonstrate that Forge can improve Mean Average Precision by 78.6% over the state-of-the-art average in the challenging multi-tab open-world scenario.

CCS Concepts

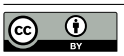
• **Security and privacy → Pseudonymity, anonymity and untraceability**; *Privacy-preserving protocols.*

Keywords

Website Fingerprinting, Multi-tab Attack, Tor, Privacy

ACM Reference Format:
Yitan Huang, Wei Qiao, Ding Wang, Meng Shen, Di Zhao, Linxu Li, Susu Cui, Bo Jiang, Zhigang Lu, and Baoxu Liu. 2026. Forge: A Robust Multi-tab Website Fingerprinting Attack Via Blind Source Separation. In *Proceedings of*

Yitan Huang and Wei Qiao contributed equally to this research.
Corresponding author: Susu Cui.



This work is licensed under a Creative Commons Attribution 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792410

the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3774904.3792410

1 Introduction

Tor [13], the leading anonymity system, provides protection against traffic analysis [26, 29, 33]. However, this strong anonymity is a double-edged sword. While safeguarding benign users, it is also exploited for malicious activities [14] , such as hosting darknet markets via its Hidden Services [37]. Consequently, adversaries are highly motivated to develop traffic analysis attacks that can bypass Tor's protections. Website Fingerprinting (WF) has emerged as a primary attack in this domain. By analyzing side-channel features of encrypted traffic—like packet sizes, timings, and directions [14, 37, 44]—WF enables a local adversary to infer a user's browsing activity, thereby undermining Tor's anonymity [2, 7, 30, 32, 34].

The field of WF has evolved significantly, moving from idealized, single-tab scenarios to the far more challenging multi-tab environment. Early methods like DF [34] and AWF [31] achieve high accuracy in these sterile, single-tab settings. However, their efficacy plummets in the complex, realistic environment of concurrent browsing. To contend with the mixed traffic from multiple open tabs, distinct strategies have emerged. Some, like ARES [12], frame the task as a multi-label classification problem, while others, like TMWF [20], treat it as an ordered set prediction. Despite these advancements, several critical challenges persist, undermining the real-world efficacy of all existing attacks:

- **The Multi-tab Interference:** Traffic from concurrently browsed websites is interleaved into a single mixed stream, obscuring individual website fingerprints and making them hard to classify.
- **The Unrealistic Pre-knowledge Requirement:** Most existing methods require prior knowledge of the exact number of open tabs. This assumption is unrealistic, and it causes a sharp performance decline when violated.
- **The Lack of Robustness and Noise Resilience:** Current models are trained on idealized datasets and largely ignore real-world factors like network noise and WF defenses. This makes their reported performance unreliable in practice.

We contend that these limitations stem from a fundamental paradigm mismatch: a problem that is intrinsically about "mixing and separation" has been rigidly modeled as multi-label classification. Imagine you are at a noisy party where many people are speaking simultaneously. Your brain does not attempt to parse the entire blended sound at once. Instead, it first separates the jumbled audio into distinct voices from the different speakers. Only after this separation can you recognize what each person is saying. This scenario, known as the classic "cocktail party problem", inspires our reformulation of multi-label website fingerprinting.

In multi-tab browsing, a single network channel similarly carries multiple interwoven "voices"—namely, the traffic patterns of each website. Applying a classifier directly to this mixed traffic forces the model to simply memorize various "combination patterns," a fragile strategy akin to recognizing the jumbled party noise as a whole rather than the individual speakers. This is not just inefficient. It is fundamentally brittle and ill-equipped to handle the dynamics of real-world browsing, such as concurrent page loads, a variable number of tabs, or network defenses. Inspired by this analogy, we turn to the solution for the classic "cocktail party problem": Blind Source Separation (BSS) [5]. BSS is a family of methods designed to computationally "unmix" such mixtures without prior knowledge of the number of sources. This insight reframes our core challenge from classifying a mixture to separating its components.

We therefore recast the multi-tab WF problem through the lens of a "separate-then-identify" paradigm: "unmixing" the traffic stream to recover the constituent website fingerprints, and subsequently identifying them. This separation-first approach obviates the need for prior knowledge of tab counts, enhances robustness against dynamic settings and defenses, and yields more stable fingerprints for downstream classification.

In this paper, we propose Forge, a framework for multi-tab WF attacks with three modules: (1) a Time-Delay Embedding module that maps one-dimensional traffic into a multi-dimensional state-space to construct virtual observation channels; (2) a BSS solver, implemented as a deep network unrolling a classical iterative BSS algorithm, which separates mixed traffic without prior knowledge of the number of concurrent tabs; and (3) a Website Identifier that classifies each separated component using Dual-Domain Consistency Attention (DDCA) to enforce time–frequency consistency.

We evaluate Forge against seven state-of-the-art methods on WFLib [11], a comprehensive dataset spanning multi-tab, open-world, and defense-enabled settings. Forge consistently achieves the best performance and demonstrates strong robustness under established defenses (e.g., WTF-PAD [23] and Front [16]), network noise, and dynamic conditions such as unknown tab counts. Extensive ablation and sensitivity analyses further isolate the contribution of each component and confirm the framework's stability.

The primary contributions of our work are as follows:

- We propose Forge, a novel framework for multi-tab WF attacks. Our framework innovatively reframes the task as a Blind Source Separation (BSS) problem. As a result, Forge can blindly separate and identify websites from a mixed traffic stream, even without prior knowledge of concurrent tab numbers.
- We design a BSS Solver by unrolling an iterative BSS algorithm into a deep neural network. This module blindly separates an unknown number of sources from a single observation.
- We develop a defense-resilient identifier equipped with the Dual-Domain Consistency Attention. The mechanism allow Forge to maintain high efficacy by validating features across domains and adaptively mitigating the impact of noise and defensive measures.
- We conduct extensive experiments on the WFlib datasets, demonstrating that Forge significantly outperforms SOTA attacks. The results, spanning closed-world, open-world, and various defense-enabled scenarios, confirm its stronger performance, generality, and robustness in realistic environments.

Availability. Our open-source implementation is available at https://github.com/Carm3nc1ta/FORGE.

2 Related Work and Limitations

2.1 Related Work

WF Attacks. Based on the scenarios they can handle, WF attacks can be divided into two categories.

Table 1: Comparison of WF attacks.

WF Attack	Multi-tab	Genericity	Robustness	Noise Resilience
AWF	✗	✗	✗	✗
DF	✗	✗	✓	✗
Tik-Tok	✗	✗	✓	✗
RF	✗	✗	✓	✗
BAPM	✓	✗	✗	✗
ARES	✓	✓	✓	✗
TMWF	✓	✓	✗	✗
FORGE (ours)	✓	✓	✓	✓

The first category is single-tab attacks, which assume that a user visits one website at a time [1, 27, 39]. Early work relies on traditional machine learning models, such as SVM in CUMUL [28, 42] and k-NN in k-FP [8, 22]. More recent studies adopt deep learning approaches, including Var-CNN [2], AWF [31], DF [34], and Tik-Tok [30], achieving improved performance.

The second category is multi-tab attacks, which reflect more realistic browsing with concurrent tabs. Early split-and-classify methods perform poorly due to information loss from discarding interleaved traffic [9, 43]. Recent work favors end-to-end models that process mixed traffic directly. TMWF [20] formulates the task as ordered set prediction using a Transformer, BAPM [18] groups traffic blocks via attention to handle overlap, and ARES [12] models the problem as multi-label classification using multiple classifiers.

WF Defenses. Concurrently, researchers develop a range of WF defenses to thwart these attacks. These defenses fall into three main categories based on their core mechanism. The first category is obfuscation. This method disrupts traffic patterns by injecting noise. Front [16] is an key example. The second is regularization[3, 4, 17]. It aims to reshape traffic from different websites into a uniform pattern. Representative works include WTF-PAD [23], Walkie-Talkie [40] and RegulaTor [19]. The third is traffic splitting. This technique compromises a fingerprint's integrity by routing it over multiple network paths. TrafficSliver [10] uses this approach.

2.2 Limitations of Existing Solutions

Despite recent progress, existing WF attacks exhibit critical limitations that hinder their real-world applicability. Our model, FORGE, is designed to address these issues, as summarized in Table 1.

Limited Multi-tab Capability: Realistic browsing involves concurrent access to multiple websites, requiring WF attacks to handle mixed traffic. However, most existing methods focus on the simpler single-tab setting and fail to support multi-tab scenarios.

Lack of Generality: Most multi-tab attacks rely on the impractical assumption that the number of open webpages is known in advance. This information is unavailable in real-world settings. ARES partially relaxes this assumption but still suffers from degraded performance without prior knowledge.

Poor Robustness to Defenses: Many influential WF attacks (e.g., AWF) are developed and evaluated without defenses. Their performance degrades significantly under standard Tor defenses such as WTF-PAD, limiting practical deployment.

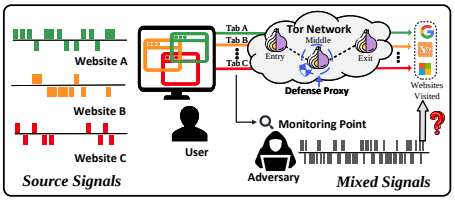
Vulnerability to Network Noise: Tor's multi-hop routing and queuing introduce unavoidable traffic noise (e.g., timing jitter). Most existing models, however, are designed for idealized, noise-free conditions and fail to account for this aspect of the Tor environment.

3 Threat Model and Motivation

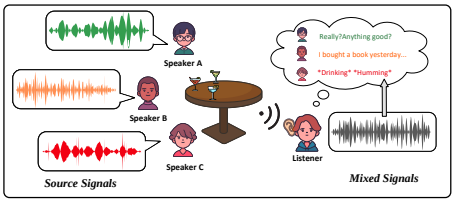
3.1 Threat Model

Figure 1a illustrates the WF threat model in the Tor network. A user, following natural browsing behavior, concurrently opens multiple webpages, producing mixed and temporally interleaved encrypted traffic. This traffic may be protected by defenses deployed on the client or Tor relays. Consistent with prior work [22, 27, 34, 36], we consider a local, passive adversary (e.g., an ISP or malicious Tor entry node [28]) that can observe encrypted traffic metadata but cannot decrypt contents or modify traffic. We consider two attack scenarios. In the closed-world setting, users visit only a small monitored set of websites. In the open-world setting, users may visit arbitrary websites, and the adversary has training data for only a subset [21, 38].

3.2 Motivation



(a) The threat model of a WF attack. An adversary attempts to deconstruct a user's browsing history by analyzing the mixed traffic from their simultaneous multi-tab browsing.



(b) The diagram of a classic "cocktail party problem". A listener can decipher the mixed audio from multiple simultaneous speakers and determine who said what.

Figure 1: The analogy between multi-tab WF attack(a) and the classic cocktail party problem(b).

Figure 1a illustrates the core challenge of multi-tab WF attack: How can an attacker deconstruct a mixed traffic stream to identify the constituent websites, especially when the number of sites is unknown and the data is distorted by network noise and defenses?

To tackle this challenge, we draw inspiration from a classic problem in signal processing: the "cocktail party problem," depicted in Figure 1b. At a crowded party, the human auditory system can remarkably focus on a single speaker's voice amidst a cacophony of other conversations and ambient noise.

The parallels between this problem and multi-tab WF are striking. We can map the core components as follows:

- **Source Signals**: Individual speakers' voices ↔ Traffic fingerprints of individual websites.
- **Mixed Signal**: The sound captured by a listener's ears ↔ The aggregated traffic sequence captured by the adversary.

- **Mixing System**: Room acoustics and reverberation ↔ The network transmission process (e.g., queuing, routing).
- **Noise**: Ambient background noise ↔ Network noise.

This analogy is not merely conceptual; it provides a formal mathematical framework. In signal processing, the cocktail party problem is a canonical example of Blind Source Separation (BSS)—a class of algorithms designed to recover original source signals from a set of mixed observations with minimal prior knowledge.

This insight motivates us to be the first to formally define the multi-tab Website Fingerprinting attack as a Blind Source Separation (BSS) problem. Under this framework, the adversary's observed traffic sequence $x[n]$ is modeled as a multi-layered mixing process:

Firstly, K concurrently accessed websites each generate an independent source signal $s_k[n]$ with a unique statistical fingerprint. Then, each signal $s_k[n]$ undergoes a time-varying process with a channel $h_k(n, \tau)$, which models network effects like latency and jitter. Finally, all streams mix non-linearly, denoted as $\mathcal{F}_{\text{nonlin}}$, due to competition for shared network resources (e.g., relay nodes).

The final observed signal $x[n]$ is thus represented as:

$$x[n] = \mathcal{F}_{\text{nonlin}}\left(\sum_{k=1}^{K}\int h_k(n,\tau;s_j)s_k(n-\tau)d\tau, \text{Queue State}\right) + \epsilon[n] \tag{1}$$

, where $\epsilon[n]$ is additive background noise. The challenge is that only the mixed signal $x[n]$ is observed, while the number of sources K, the signals s_k, and the non-linear mixing $\mathcal{F}_{\text{nonlin}}$ are unknown. The goal is to recover the website labels $\hat{Y} = \hat{y}_1, \ldots, \hat{y}_{K'}$.

4 Design of Forge

We propose **Forge**, a framework consisting of three functionally distinct, sequential modules, as shown in Figure 2: 1) Time-Delay Embedding module; 2) BSS Solver; and 3) Website Identifier.

4.1 Traffic Channel Construction

Inspired by signal processing, we model multi-tab Website Fingerprinting as a Blind Source Separation (BSS) problem. A key challenge is that classic BSS requires multiple observation channels (M) to separate multiple sources (K), whereas in WF attacks, the adversary observes only a single mixed traffic stream with per-packet directions and timestamps. This creates an underdetermined scenario ($M = 1, K > 1$), making unique separation impossible.

To overcome this, we propose constructing virtual observation channels from a single traffic stream using Takens's theorem[6], which states that a system's dynamics can be reconstructed from a single observable. By forming state vectors of time-delayed signals, we convert the one-dimensional series into a higher-dimensional space[15, 35], generating the multiple "channels" needed for BSS.

Following this principle, we reconstruct a high-dimensional state space from our one-dimensional sequence $x[n]$ with length l, representing the sequence of encrypted packets over time. Each element in this sequence encodes critical side-channel information, primarily the packet's direction (+1 for outgoing, -1 for incoming) and its timestamp. A state vector $v[t]$ at time t is constructed by applying a mapping operator Φ to the signal's history over a duration τ: $v[t] = \Phi(x[t-\tau : t])$. Here, $x[t-\tau : t]$ denotes the traffic segment in the preceding time window. The operator $\Phi : \mathcal{X} \to \mathbb{R}^8$ maps this segment to an 8-dimensional vector, whose components are defined as follows:

Volumetric Dynamics: Let $D(x[t-\tau : t])$ be the sequence of packet directions (+1 for outgoing, -1 for incoming) in the window, and $I(\cdot)$ be the indicator function.

- $v_0[t] = \sum_{d_i \in D(x[t-\tau:t])} I(d_i > 0)$;
- $v_1[t] = \sum_{d_i \in D(x[t-\tau:t])} I(d_i < 0)$.

Temporal Structure: Let T_{up} and T_{down} be the sets of sorted timestamps for outgoing and incoming packets within the window, respectively.

- $v_2[t] = \sum_i (\text{ts}_{i+1} - \text{ts}_i)$ for $\text{ts}_i, \text{ts}_{i+1} \in T_{\text{up}}$;
- $v_3[t] = \sum_j (\text{ts}'_{j+1} - \text{ts}'_j)$ for $\text{ts}'_j, \text{ts}'_{j+1} \in T_{\text{down}}$.

Interaction Patterns: A "Burst" is defined as a contiguous subsequence of the same sign in $D(x[t-\tau : t])$. Let B_{up} and B_{down} be the sets of outgoing and incoming bursts, respectively.

- $v_4[t] = |B_{\text{up}}|$;
- $v_5[t] = |B_{\text{down}}|$;
- $v_6[t] = (1/|B_{\text{up}}|) \sum_{b \in B_{\text{up}}} \text{len}(b)$;
- $v_7[t] = (1/|B_{\text{down}}|) \sum_{b \in B_{\text{down}}} \text{len}(b)$.

By applying this operator over the entire observation duration, we elevate the original 1D sequence $x[n]$ into an 8-dimensional state-vector time series $V = [v, v, \ldots, v[T]] \in \mathbb{R}^{8\times T}$. This step is equivalent to creating $M_{\text{virtual}} = 8$ virtual observation channels, providing a mathematically well-posed input for the subsequent blind source separation.

4.2 BSS Solver

The BSS solver disentangles multi-channel features $\mathbf{V} \in \mathbb{R}^{8\times T}$ into single-website source streams $\hat{\mathbf{S}}$. Leveraging Blind Source Separation (BSS), it operates without knowing the number of sources, making the model robust when the number of concurrently open websites (K) is unknown.

The solver mimics iterative BSS algorithms, which refine mixed signals until statistically independent. We implement this via a deep network of L cascaded **Iterative Correction Modules (ICMs)**, each performing one separation step. At iteration l, the ICM takes the current estimate $\mathbf{H}^{(l)}$ and computes a correction $\mathcal{F}$, producing a refined estimate $\mathbf{H}^{(l+1)}$:

$$\mathbf{H}^{(l+1)} = \text{PostProcess}(\mathbf{H}^{(l)} + \mathcal{F}(\mathbf{H}^{(l)})) \tag{2}$$

.The function $\mathcal{F}$ represents the core computational block designed to compute and apply this correction. Its architecture is detailed below.

Step 1: Residual Feature Extraction. This step aims to analyze the current signal estimate $\mathbf{H}^{(l)}$ to identify residual cross-stream dependencies.

- *Learnable Analysis Filters*: The input $\mathbf{H}^{(l)}$ is processed by a Conv1d layer and BatchNorm1d, acting as learnable filters that project the signal into a feature space where residual mixing patterns can be more effectively identified.
- *Non-linear Activation*: The projected features pass through a ReLU, promoting sparsity by zeroing negative values and helping the model separate the original source signals from the mixture.

Step 2: Correction Signal Synthesis. The non-linearly transformed features, which now highlight the distinct source characteristics, are fed into a second pair of Conv1d and BatchNorm1d layers.

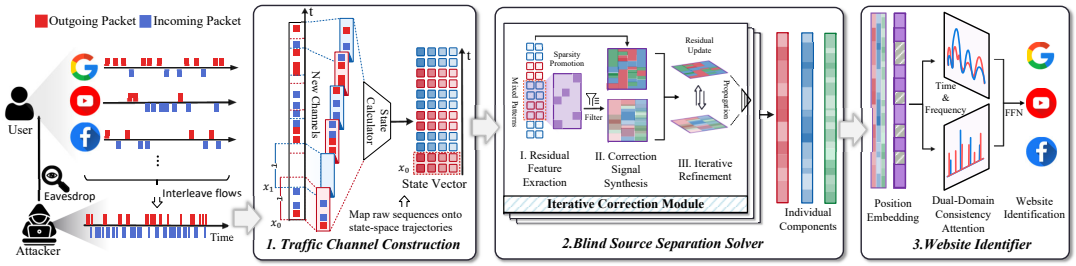


Figure 2: The architecture of Forge. The framework consists of three main stages: 1.Traffic Channel Construction module converts raw traffic into a multi-dimensional representation. 2. BSS Solver separates the mixed traffic into individual components. 3. Website Identifier classifies each separated component using Dual-Domain Consistency Attention (DDCA).

This stage acts as a synthesis filter bank, converting the extracted residual features back into a correction signal, $\Delta\mathbf{H}$, which matches the dimensions of the input.

Step 3: Iterative Refinement. This final step applies the correction and prepares the output for the next module in the cascade.

- *Residual Update*: The synthesized correction $\Delta\mathbf{H}$ is added to the module's input $\mathbf{H}^{(l)}$ via a shortcut, implementing the iterative update rule new_estimate = old_estimate + update_term. This residual design lets each ICM focus on learning the correction, simplifying optimization.
- *Multi-scale Feature Propagation*: The updated signal passes through a ReLU activation and a MaxPool1d layer for down-sampling, enabling the next ICM to capture features from local patterns to long-term dependencies. A Dropout layer is applied for regularization before the output is passed on.

By cascading L such ICMs, the network achieves progressive, multi-scale separation. The final output $\mathbf{H}^{(L)}$ comprises refined feature streams, with each channel conditioned to represent a distinct underlying source signal.

4.3 Website Identifier

This module performs the final classification using a Transformer-based identifier with Dual-Domain Consistency Attention (DDCA). Learnable Positional Embeddings are added to each stream before the encoder, whose core comprises stacked DDCA blocks.

Dual-Domain Consistency Attention (DDCA). A key challenge in website fingerprinting is adversarial defenses. As shown in Figure 3, methods like dummy-packet injection and adaptive padding mainly disrupt time-domain features such as packet timing and sequence length. These perturbations are seldom "clean" and often leave artifacts in the frequency domain—for instance, rapid dummy-packet injection generates high-frequency noise, while other padding schemes produce predictable frequency signatures.

DDCA is motivated by the observation that time-domain obfuscation often leaves traces in the frequency domain. It employs a dual-branch attention mechanism to cross-validate features from both domains, enhancing robustness against defenses targeting only the time domain.

Time-Domain Dependency Analyzer: We use a standard self-attention mechanism to model sequential dependencies in the time dimension, capturing the local temporal logic between events. Its

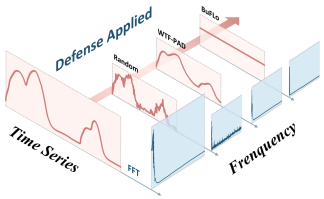


Figure 3: Time–frequency views of three examples, one per defense category: random injection, adaptive padding, and regularization.

computation is:

$$y_{\text{time}} = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{3}$$

, where Q, K, and V are linear projections of the input x.

Frequency-Domain Structure validator: This branch is designed to capture frequency-domain features and validate them against time-domain noise. To achieve this, the time-domain signal x is first mapped to the frequency domain using the Fast Fourier Transform:

$$X_f[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i2\pi kn/N} \tag{4}$$

. Crucially, we use only the magnitude spectrum $A_f = |X_f|$, which is naturally robust to time shifts and jitter introduced by defenses.

Self-attention is then applied to this magnitude spectrum, enabling the model to focus on stable low-to-mid frequency bands that reflect a site's intrinsic loading rhythm, while down-weighting high-frequency noise characteristic of certain defenses.

The time and frequency branch outputs are fused via a residual connection gated by a learnable parameter $\alpha = 0.1$. After the DDCA module, the signal passes through a feed-forward network (FFN), and a final linear layer produces the classification logits, yielding the multi-website fingerprinting labels.

5 Evaluation

In this section, we present various evaluations to demonstrate the merits of Forge and answer the following key research questions:
RQ1: How does Forge perform in a closed-world, multi-tab setting?
RQ2: How effective is Forge in an open-world, multi-tab setting?

Table 2: Performance of WF Attacks across different tab number settings in closed world.

Attacks	$D_{\text{CW-2tab}}$			$D_{\text{CW-3tab}}$			$D_{\text{CW-4tab}}$			$D_{\text{CW-5tab}}$		
	Pre	Recall	F1-score	Pre	Recall	F1-score	Pre	Recall	F1-score	Pre	Recall	F1-score
AWF	0.4864	0.0139	0.0270	0.3450	0.0040	0.0078	0.4691	0.0033	0.0065	0.3750	0.0013	0.0027
DF	0.8482	0.4838	0.6162	0.8419	0.2604	0.3978	0.8306	0.1747	0.2886	0.7526	0.0385	0.0732
Tik-Tok	0.8506	0.3868	0.5318	0.8321	0.1595	0.2677	0.8409	0.135	0.2326	**0.8508**	0.0944	0.1699
RF	0.6604	0.3983	0.4969	0.7387	0.5953	0.6593	0.6700	0.3390	0.4502	0.5598	0.2371	0.3331
BAPM	0.7837	0.3665	0.4994	0.7098	0.1951	0.3061	0.6871	0.1578	0.2567	0.6573	0.0971	0.1693
TMWF	0.7821	0.6702	0.7218	0.7043	0.5717	0.6311	0.6793	0.5561	0.6115	0.6202	0.4599	0.5282
ARES	0.8285	0.7011	0.7595	0.7971	0.6022	0.6861	0.8351	0.5930	0.6935	0.7619	0.3455	0.4754
FORGE	**0.8527**	**0.7758**	**0.8124**	**0.8434**	**0.7378**	**0.7871**	**0.8430**	**0.7434**	**0.7900**	0.8027	**0.7336**	**0.7666**

- **Bold** denotes the best results, and underline denotes the second-best results.

RQ3: How robust is FORGE against established WF defenses?
RQ4: How does FORGE perform in dynamic environments?
RQ5: How does FORGE perform under network noise?
RQ6: What is the contribution of each core component to FORGE 's overall efficacy?

5.1 Experiment Setup

5.1.1 Datasets. We evaluate FORGE on publicly available and modified datasets (details in Appendix B).
Closed- and Open-world: Sourced from WFLib [11], the closed-world dataset D_{CW} covers Alexa Top 100 sites and is partitioned by concurrent tabs into $D_{\text{CW-2tab}}$–$D_{\text{CW-5tab}}$. The open-world dataset D_{OW} includes unmonitored sites and is partitioned similarly.
Defense Datasets: Six practical defenses are evaluated. Open-source datasets $D_{\text{WTF-PAD}}$, D_{Front}, $D_{\text{Walkie-Talkie}}$, and $D_{\text{TrafficSliver}}$ come from WFLib, while D_{Random}, $D_{\text{RegularTor-Light}}$, and $D_{\text{RegularTor-Heavy}}$ are constructed from $D_{\text{CW-2tab}}$ (see Appendix E).
Dynamic Datasets: For dynamic evaluation, we sample and mix instances across tab counts and defense schemes to form $D_{\text{mix-num-tabs}}$ and $D_{\text{mix-defenses}}$ (details in Appendix B).
Noise Datasets: Random noise is injected into $D_{\text{CW-2tab}}$ to create D_{Noise}; parameters and strategies are detailed in Appendix F.

5.1.2 Baselines. To assess FORGE, we compare it with seven established WF methods across three categories. For readability, figures show five baselines, omitting the oldest from each of the last two categories. See the Appendix D for their results. (1) Single-tab WF attacks: AWF [31], which applies a classic CNN to packet-direction sequences. (2) Defense-resilient single-tab WF attacks: DF [34], a defense-aware CNN; Tik-Tok [30], which fuses directional and temporal features; and RF [32], which constructs a 2D feature matrix. (3) Multi-tab WF attacks: BAPM [18], which uses block segmentation with attention-based association for joint identification; ARES [12], a Transformer ensemble robust to obfuscation; and TMWF [20], which extract per-tab embeddings for joint recognition.

To ensure a fair comparison, all baseline models are adapted to perform multi-label classification. Specific details of these modifications are provided in Appendix C.

5.1.3 Metrics. **Multi-Label Metrics:** We use AUC [24], P@K and MAP@K [25]. For an instance, let y be the ground-truth label vector and $\hat{y}$ be the predicted probability vector. P@K (Precision at K) measures the proportion of correct labels within the top-k predictions. Let $r_k(\hat{y})$ be the set of labels with the top-k highest probabilities.

$$P@k = \frac{1}{k} \sum_{l \in r_k(\hat{y})} y_l. \tag{5}$$

MAP@K (Mean Average Precision at K) extends P@K by also considering the rank of the correct predictions, thus rewarding correctly identified labels that appear higher in the list.

$$MAP@k = \frac{\sum_{i=1}^{k} P@i}{k}. \tag{6}$$

Single-Label Metrics: We report standard metrics including Precision, Recall, and F1-Score, computed from true positives, false positives, true negatives, and false negatives.

5.1.4 Implementation. Our implementation of FORGE is developed in PyTorch 2.4.1 and Python 3.10.5 consisting of over 1,500 lines of code. All experiments were performed on a server with an Intel(R)Xeon(R)Silver 4208 CPU. The hyperparameter configuration is detailed in Appendix A.

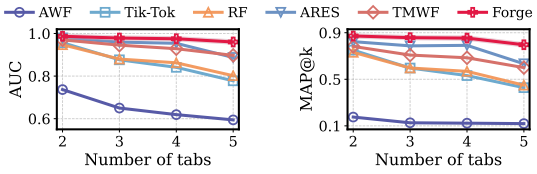


Figure 4: Performance comparison of different methods on varying tab number settings.

5.2 Closed-world Evaluation(RQ1)

We evaluate FORGE on closed-world datasets with varying tab counts using multiple metrics. Figure 4 shows AUC and MAP@k across tab settings in D_{CW}. FORGE outperforms all baselines in AUC, with single-page methods failing on multi-tab data and native multi-tab methods performing better; FORGE remains stable even at 5 tabs. MAP@k, with k equal to the tab count, shows similar gains, with FORGE consistently surpassing all WF attacks.

We further evaluate top-k predictions using Precision@k (P@k), the fraction of true sites within the top-k. Figure 5 shows FORGE outperforms alternatives across tab counts. Its advantage grows with more tabs; on $D_{\text{CW-5tab}}$ it still achieves high P@2, indicating a higher likelihood of recovering the user's true sites.

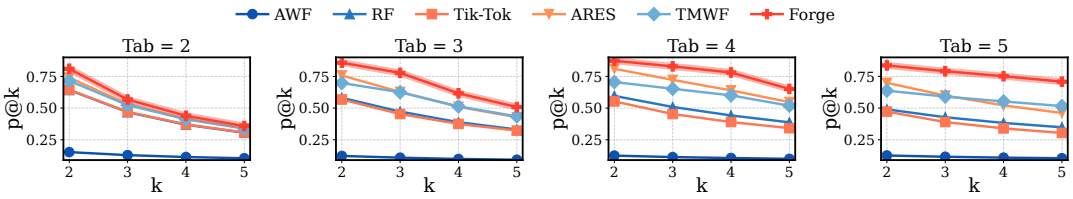


Figure 5: Comparison of Precision@k for different methods on varying tab number settings.

Table 2 shows that FORGE achieves the best results across almost all metrics. Tik-Tok has higher precision but much lower recall on $D_{\text{CW-5tab}}$, meaning it predicts very few positives and misses many true labels. Therefore, F1-score is the most appropriate metric here. Higher F1-score indicates a better balance and better performance.

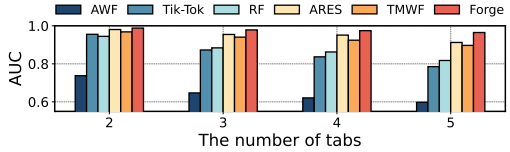


Figure 6: Comparison of AUC performance on the open-World dataset across different tab settings.

5.3 Open-world Evaluation(RQ2)

We extended our evaluation to an open-world scenario, classifying unmonitored websites into a single class. As Table 3 reports, FORGE again demonstrated its superiority, achieving a 78.6% improvement of MAP@5 over the average. Furthermore, its AUC scores are the highest, approaching 1.0 (2-tab) and holding at 0.98 (5-tab), as shown in Figure 6. This confirms FORGE's ability to achieve SOTA accuracy under growing attack complexity.

5.4 FORGE Against Defenses(RQ3)

We evaluate FORGE 's adversarial robustness in multi-tab settings against four WF defenses (Table 4). FORGE consistently outperform all baselines. Against obfuscation defenses (Random, Front, WTF-PAD), it shows strong resilience with AUCs of 0.985, 0.9996, and 0.999, respectively. Against the heavyweight defense RegularTor, FORGE's MAP@k dropped but still surpassed all baselines, though such defenses are noted as impractical due to overhead. In additional single-tab tests against Walkie-Talkie and TrafficSliver, FORGE again achieved the best results (Figure 7). We attribute this robustness to its architecture's joint exploitation of time- and frequency-domain cues, which captures residual site-specific patterns that persist after defense.

5.5 Dynamic Settings(RQ4)

We evaluate FORGE's genericity in two dynamic scenarios: unknown tab numbers and unknown WF defenses. As shown in Table 5, FORGE achieves an AUC of 0.985 in the multi-tab scenario. In the defense scenario, it achieves an AUC of 0.999. This genericity extends to mismatched training (Figure 8). For instance, a model trained on 4-tab data achieves a high AUC of 0.85 on 3-tab data, confirming its effectiveness without precise prior knowledge.

Table 3: WF attack performance across varying tab numbers.

Datasets	Attacks	Pre	AP@k	MAP@k
$D_{\text{OW-2tab}}$	AWF	0.4988 ▼42.3%	0.1587 ▼80.5%	0.1826 ▼79.2%
	DF	0.8509 ▼1.6%	0.5830 ▼28.5%	0.6923 ▼21.2%
	Tik-Tok	**0.8832** ▲2.2%	0.6305 ▼22.7%	0.7368 ▼16.1%
	RF	0.7534 ▼12.9%	0.6297 ▼22.8%	0.7155 ▼18.5%
	BAPM	0.7800 ▼9.8%	0.5216 ▼36.0%	0.6134 ▼30.2%
	TMWF	0.7868 ▼9.0%	0.6891 ▼15.5%	0.7597 ▼13.5%
	ARES	0.8427 ▼2.5%	0.7448 ▼8.7%	0.8239 ▼6.2%
	FORGE	0.8646 ▲12.2% [1]	**0.8155** ▲44.2%	**0.8782** ▲35.9%
$D_{\text{OW-3tab}}$	AWF	0.3733 ▼55.2%	0.1123 ▼85.5%	0.1292 ▼84.9%
	DF	0.8451 ▲1.4%	0.4222 ▼45.5%	0.5637 ▼34.2%
	Tik-Tok	**0.8591** ▲3.0%	0.4483 ▼42.1%	0.5946 ▼30.6%
	RF	0.6850 ▼17.8%	0.4823 ▼37.7%	0.6105 ▼28.8%
	BAPM	0.7349 ▼11.9%	0.3830 ▼50.5%	0.4895 ▼42.9%
	TMWF	0.6868 ▼17.6%	0.6087 ▼21.4%	0.6940 ▼19.0%
	ARES	0.8163 ▼2.1%	0.6438 ▼16.8%	0.7706 ▼10.1%
	FORGE	0.8338 ▲16.7%	**0.7740** ▲74.7%	**0.8569** ▲55.7%
$D_{\text{OW-4tab}}$	AWF	0.4701 ▼43.2%	0.1122 ▼85.4%	0.1317 ▼84.5%
	DF	0.8528 ▲3.1%	0.3701 ▼51.9%	0.5069 ▼40.2%
	Tik-Tok	**0.8839** ▲6.8%	0.3859 ▼49.9%	0.5285 ▼37.6%
	RF	0.6713 ▼18.9%	0.4383 ▼43.1%	0.5655 ▼33.3%
	BAPM	0.6641 ▼19.7%	0.3435 ▼55.4%	0.4395 ▼48.1%
	TMWF	0.6737 ▼18.6%	0.5875 ▼23.7%	0.6688 ▼21.1%
	ARES	0.8129 ▼1.7%	0.6572 ▼14.7%	0.7775 ▼8.3%
	FORGE	0.8273 ▲15.2%	**0.7702** ▲86.3%	**0.8476** ▲64.0%
$D_{\text{OW-5tab}}$	AWF	0.4318 ▼46.8%	0.1069 ▼85.4%	0.1234 ▼84.9%
	DF	0.8528 ▲5.1%	0.3152 ▼56.9%	0.4475 ▼45.3%
	Tik-Tok	**0.8837** ▲8.9%	0.3167 ▼56.7%	0.4471 ▼45.3%
	RF	0.5948 ▼26.7%	0.3713 ▼49.2%	0.4835 ▼40.9%
	BAPM	0.6168 ▼24.0%	0.3092 ▼57.7%	0.3984 ▼51.3%
	TMWF	0.6375 ▼21.5%	0.5172 ▼29.3%	0.6060 ▼25.9%
	ARES	0.7831 ▼3.5%	0.5560 ▼24.0%	0.7003 ▼14.4%
	FORGE	0.8116 ▲18.3%	**0.7312** ▲105.4%	**0.8181** ▲78.6%

- **Bold** denotes the best, and underline denotes the second-best.
▼%= % reduction, ▲% = % improvement.
[1] Improvement over other methods' average.

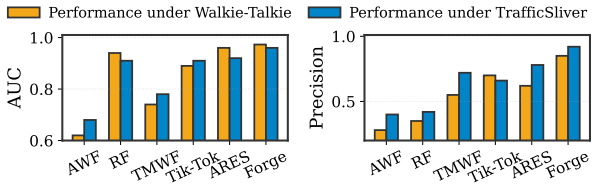


Figure 7: Comparison of AUC and precision under Walkie-Talkie and TrafficSliver on the single-tab setting.

Table 4: WF attack performance under four representative defenses on the 2-tab setting.

Attacks	Random			Front			WTF-PAD			RegularTor-Light			RegularTor-Heavy		
	Pre	AUC	MAP@k	Pre	AUC	MAP@k	Pre	AUC	MAP@k	Pre	AUC	MAP@k	Pre	AUC	MAP@k
AWF	0.4693	0.7246	0.1381	0.6946	0.9224	0.2991	0.6855	0.9610	0.4431	0.0000	0.5509	0.0363	0.0000	0.5530	0.0342
DF	0.8183	0.9181	0.6081	0.8791	0.9949	0.6423	0.9242	0.9987	0.6980	0.7640	0.7778	0.3126	**0.7935**	0.7477	0.2434
Tik-Tok	**0.8665**	0.9414	0.6859	0.9275	0.9983	0.6898	0.9592	0.9994	0.7165	**0.7776**	0.8035	0.3595	0.7717	0.7647	0.2638
RF	0.7743	0.9416	0.7030	0.9557	0.9967	0.7054	0.9672	0.9978	0.7182	0.0000	0.5051	0.0272	0.0000	0.5034	0.0273
BAPM	0.7407	0.9139	0.5253	0.8293	0.9817	0.5277	0.8617	0.9944	0.6460	0.6662	0.8129	0.2795	0.6466	0.7935	0.2357
TMWF	0.7240	0.9574	0.6999	0.8808	0.9942	0.6318	0.9306	0.9978	0.6909	0.6260	0.9040	0.4850	0.7173	0.8754	0.4011
ARES	0.7890	0.9714	0.7659	0.9428	0.9994	0.7136	0.9426	0.9993	0.7063	0.7293	0.9342	0.5775	0.7123	0.9187	0.5149
FORGE	0.8500	**0.9848**	**0.8536**	**0.9641**	**0.9996**	**0.7220**	**0.9713**	**0.9998**	**0.7294**	0.7195	**0.9486**	**0.6467**	0.7270	**0.9357**	**0.5827**

- **Bold** denotes the best results, and underline denotes the second-best results.

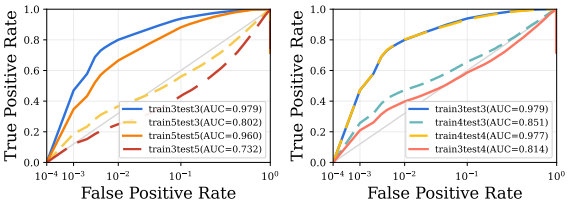
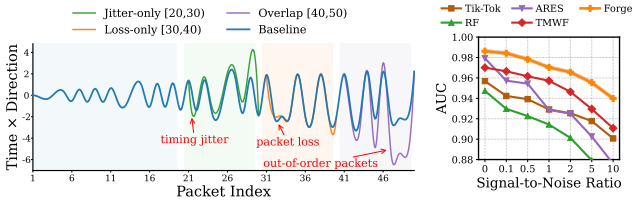


Figure 8: AUC curve of FORGE when the number of tabs differs between training and testing.

Table 5: Performance under Mix tab settings and Mix defenses.

Attacks	$D_{\text{mix-num-tabs}}$			$D_{\text{mix-defenses}}$		
	Pre	AUC	AP@5	Pre	AUC	AP@2
AWF	0.5070	0.6515	0.1316	0.4993	0.8949	0.2344
DF	0.9034	0.8264	0.4642	0.8127	0.9896	0.5840
Tik-Tok	**0.9254**	0.8438	0.4977	0.8538	0.9951	0.6401
RF	0.7780	0.8563	0.5192	**0.9472**	0.9924	0.6778
BAPM	0.7598	0.8426	0.4171	0.6969	0.9742	0.4776
TMWF	0.7488	0.9176	0.5757	0.8184	0.9889	0.5805
ARES	0.8518	0.9646	0.7071	0.7873	0.9923	0.5865
FORGE	0.8500	**0.9848**	**0.8536**	0.9192	**0.9986**	**0.6892**

- **Bold** denotes the best, and underline denotes the second-best.



(a) Waveform Changes. **(b) AUC performance.**

Figure 9: Analysis of Noise Interference Effects.(a) Changes in the $Time \times Direction$ waveform under different noise interferences; (b) AUC of different models across noise intensities.

5.6 FORGE Against Noise(RQ5)

We evaluate FORGE's noise robustness by simulating packet loss, timing jitter, and out-of-order packets [41] at levels from 0.1% to 10%. AWF is omitted due to severe degradation (see Appendix D). As shown in Figure 9b, FORGE significantly outperforms baselines, maintaining an AUC above 0.9 even at 10% noise. This robustness stems from precise temporal modeling in the channel construction module and the DDCA module's joint time–frequency analysis.

Table 6: Ablation analysis of FORGE.

Test Case	Time Delay Embedding	BSS Solver	Website Identifier	P@2	MAP@2
Case_1	✗	✓	✓	0.7253 ▼9.5%	0.7889 ▼8.7%
Case_2	✓	✗	✓	0.3478 ▼56.6%	0.4114 ▼52.4%
Case_3	✓	✓	✗	0.7012 ▼12.5%	0.8034 ▼7.0%
Case_4	✓	✓	✓	**0.8016**	**0.8640**

- ▼%= % reduction.

5.7 Ablation Study(RQ6)

We conduct an ablation study on $D_{\text{CW-2tab}}$ to evaluate FORGE 's core components. Table 6 shows that removing any module significantly degrades performance: replacing state vectors with raw sequences reduces MAP@2 by 7.51%; removing the BSS solver causes a larger 45.26% drop; and substituting the specialized identifier with a linear model lowers MAP@2 from 0.864 to 0.803.

6 Conclusion

In this paper, we introduce FORGE, a multi-tab WF attack that reframes the task as a BSS problem. This shift enables FORGE to blindly identify an unknown number of websites from mixed traffic streams. FORGE comprises a novel BSS solver for traffic demixing and a robust identifier with Dual-Domain Consistency Attention to withstand WF defenses and network noise. We conduct comprehensive evaluations across multiple datasets and settings, demonstrating that FORGE consistently surpasses prior state-of-the-art attacks in closed- and open-world regimes, under defense-enabled conditions, and in other challenging, real-world scenarios.

Acknowledgments

This research is supported by National Key Research and Development Program of China (No.2023YFC2206402), the Strategic Priority Research Program of the Chinese Academy of Sciences (No.XDA046 0100), and Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China (No.KLCS20240206). This work is also supported by the Program of Key Laboratory of Network Assessment Technology, the Chinese Academy of Sciences, Program of Beijing Key Laboratory of Network Security and Protection Technology.

References

[1] Alireza Bahramali, Ardavan Bozorgi, and Amir Houmansadr. 2023. Realistic website fingerprinting by augmenting network traces. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1035–1049.

[2] Sanjit Bhat, David Lu, Albert Kwon, and Srinivas Devadas. 2019. Var-CNN: A Data-Efficient Website Fingerprinting Attack Based on Deep Learning. *Proceedings on Privacy Enhancing Technologies* 2019 (10 2019), 292–310. doi:10.2478/popets-2019-0070

[3] Xiang Cai, Rishab Nithyanand, and Rob Johnson. 2014. Cs-buflo: A congestion sensitive website fingerprinting defense. In *Proceedings of the 13th Workshop on Privacy in the Electronic Society*. 121–130.

[4] Xiang Cai, Rishab Nithyanand, Tao Wang, Rob Johnson, and Ian Goldberg. 2014. A systematic approach to developing and evaluating website fingerprinting defenses. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 227–238.

[5] Jean-François Cardoso. 1998. Blind Signal Separation: Statistical Principles. *Proc. IEEE* 86, 10 (1998), 2009–2025. doi:10.1109/5.720250

[6] Yen-Lin Chen, Yuan Chiang, Pei-Hsin Chiu, I-Chen Huang, Yu-Bai Xiao, Shu-Wei Chang, and Chang-Wei Huang. 2021. High-Dimensional Phase Space Reconstruction with a Convolutional Neural Network for Structural Health Monitoring. *Sensors* 21, 10 (2021), 3514. doi:10.3390/s21103514

[7] Giovanni Cherubin, Rob Jansen, and Carmela Troncoso. 2022. Online website fingerprinting: Evaluating website fingerprinting attacks on tor in the real world. In *31st USENIX Security Symposium (USENIX Security 22)*. 753–770.

[8] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.

[9] Weiqi Cui, Tao Chen, Christian Fields, Julianna Chen, Anthony Sierra, and Eric Chan-Tin. 2019. Revisiting assumptions for website fingerprinting attacks. In *Proceedings of the 2019 ACM asia conference on computer and communications security*. 328–339.

[10] Wladimir De la Cadena, Asya Mitseva, Jens Hiller, Jan Pennekamp, Sebastian Reuter, Julian Filter, Thomas Engel, Klaus Wehrle, and Andriy Panchenko. 2020. Trafficsliver: Fighting website fingerprinting attacks with traffic splitting. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1971–1985.

[11] Xinhao Deng, Qi Li, and Ke Xu. 2024. Robust and Reliable Early-Stage Website Fingerprinting Attacks via Spatial-Temporal Distribution Analysis. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*.

[12] Xinhao Deng, Qilei Yin, Zhuotao Liu, Xiyuan Zhao, Qi Li, Mingwei Xu, Ke Xu, and Jianping Wu. 2023. Robust multi-tab website fingerprinting attacks in the wild. In *2023 IEEE symposium on security and privacy (SP)*. IEEE, 1005–1022.

[13] Roger Dingledine, Nick Mathewson, and Paul Syverson. 2004. Tor: The Second-Generation Onion Router. In *13th USENIX Security Symposium (USENIX Security 04)*. USENIX Association, San Diego, CA. https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router

[14] Priyanka Dodia, Mashael AlSabah, Omar Alrawi, and Tao Wang. 2022. Exposing the rat in the tunnel: Using traffic analysis for tor-based malware detection. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 875–889.

[15] William Gilpin. 2020. Deep Reconstruction of Strange Attractors from Time Series. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33.

[16] Jiajun Gong and Tao Wang. 2020. Zero-delay lightweight defenses against website fingerprinting. In *29th USENIX Security Symposium (USENIX Security 20)*. 717–734.

[17] Jiajun Gong, Wuqi Zhang, Charles Zhang, and Tao Wang. 2022. Surakav: Generating realistic traces for a strong website fingerprinting defense. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1558–1573.

[18] Zhong Guan, Gang Xiong, Gaopeng Gou, Zhen Li, Mingxin Cui, and Chang Liu. 2021. BAPM: block attention profiling model for multi-tab website fingerprinting attacks on tor. In *Proceedings of the 37th Annual Computer Security Applications Conference*. 248–259.

[19] James K Holland and Nicholas Hopper. 2020. Regulator: A straightforward website fingerprinting defense. *arXiv preprint arXiv:2012.06609* (2020).

[20] Zhaoxin Jin, Tianbo Lu, Shuang Luo, and Jiaze Shang. 2023. Transformer-based model for multi-tab website fingerprinting attack. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1050–1064.

[21] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Mario Grattafiori. 2014. A Critical Evaluation of Website Fingerprinting Attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security* (Scottsdale, Arizona, USA) *(CCS '14)*. Association for Computing Machinery, New York, NY, USA, 763–774. doi:10.1145/2660267.2660368

[22] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Mario Grattafiori. 2016. k-fingerprinting: A Robust Scalable Website Fingerprinting Technique. In *25th USENIX Security Symposium (USENIX Security '16)*. USENIX Association, Austin, TX, USA, 1139–1154.

[23] Marc Juárez, Mohsen Imani, Mike Perry, Claudia Dıaz, and Matthew Wright. 2015. WTF-PAD: toward an efficient website fingerprinting defense for tor. *CoRR, abs/1512.00524* (2015).

[24] Charles X Ling, Jin Huang, Harry Zhang, et al. 2003. AUC: a statistically consistent and more discriminating measure than accuracy. In *Ijcai*, Vol. 3. 519–524.

[25] Jingzhou Liu, Wei-Cheng Chang, Yuexin Wu, and Yiming Yang. 2017. Deep learning for extreme multi-label text classification. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*. 115–124.

[26] Se Eun Oh, Taiji Yang, Nate Mathews, James K Holland, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. 2022. Deepcoffea: Improved flow correlation attacks on tor via metric learning and amplification. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1915–1932.

[27] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinner, Martin Henze, and Klaus Wehrle. 2011. Website Fingerprinting: Attacking Popular Privacy Enhancing Technologies with the Multinomial Naïve-Bayes Classifier. In *Proceedings of the 2011 ACM SIGCOMM Conference on Internet Measurement Conference* (Berlin, Germany) *(IMC '11)*. Association for Computing Machinery, New York, NY, USA, 15–28. doi:10.1145/2068816.2068819

[28] Andriy Panchenko, Fabian Lanze, Andreas Zinnen, Martin Henze, Jan Pennekamp, Klaus Wehrle, and Thomas Engel. 2016. Website Fingerprinting at Internet Scale. doi:10.14722/ndss.2016.23477

[29] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. 2011. Website fingerprinting in onion routing based anonymization networks. In *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society*. 103–114.

[30] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Gangadhara, and Matthew Wright. 2020. Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks. *Proceedings on Privacy Enhancing Technologies* 2020 (07 2020), 5–24. doi:10.2478/popets-2020-0043

[31] Vera Rimmer, Davy Preuveneers, Marc Juarez, Tom Van Goethem, and Wouter Joosen. 2017. Automated website fingerprinting through deep learning. *arXiv preprint arXiv:1708.06376* (2017).

[32] Meng Shen, Kexin Ji, Zhenbo Gao, Qi Li, Liehuang Zhu, and Ke Xu. 2023. Subverting website fingerprinting defenses with robust traffic representation *(SEC '23)*. USENIX Association, USA, Article 35, 18 pages.

[33] Meng Shen, Kexin Ji, Jinhe Wu, Qi Li, Xiangdong Kong, Ke Xu, and Liehuang Zhu. 2024. Real-time website fingerprinting defense via traffic cluster anonymization. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3238–3256.

[34] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. 2018. Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 1928–1943. doi:10.1145/3243734.3243768

[35] Eugene Tan, Shannon Algar, Débora Corrêa, Michael Small, Thomas Stemler, and David Walker. 2023. Selecting Embedding Delays: An Overview of Embedding Techniques and a New Method Using Persistent Homology. *Chaos* 33, 3 (2023), 032101. doi:10.1063/5.0137223

[36] Carmela Troncoso, Sergej Prokhorenko, Pjotr Kopytko, Wouter Joosen, and George Danezis. 2017. Touching from a Distance: Website Fingerprinting Attacks and Defenses. *Proceedings on Privacy Enhancing Technologies* 2017, 4 (2017), 127–145. doi:10.1515/popets-2017-0040

[37] Chunmian Wang, Zhen Ling, Wenjia Wu, Qi Chen, Ming Yang, and Xinwen Fu. 2022. Large-scale evaluation of malicious tor hidden service directory discovery. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 1709–1718.

[38] Tao Wang. 2020. High precision open-world website fingerprinting. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 152–167.

[39] Tao Wang, Xiang Cai, Rishab Nithyanand, Rob Johnson, and Ian Goldberg. 2014. Effective attacks and provable defenses for website fingerprinting. In *Proceedings of the 23rd USENIX Conference on Security Symposium* (San Diego, CA) *(SEC'14)*. USENIX Association, USA, 143–157.

[40] Tao Wang and Ian Goldberg. 2017. {Walkie-Talkie}: An efficient defense against passive website fingerprinting attacks. In *26th USENIX Security Symposium (USENIX Security 17)*. 1375–1390.

[41] Renjie Xie, Yixiao Wang, Jiahao Cao, Enhuan Dong, Mingwei Xu, Kun Sun, Qi Li, Licheng Shen, and Menghao Zhang. 2023. Rosetta: Enabling robust tls encrypted traffic classification in diverse network environments with tcp-aware traffic augmentation. In *Proceedings of the ACM turing award celebration conference-China 2023*. 131–132.

[42] Hui Xue, Qiang Yang, and Songcan Chen. 2009. SVM: Support vector machines. In *The top ten algorithms in data mining*. Chapman and Hall/CRC, 51–74.

[43] Qilei Yin, Zhuotao Liu, Qi Li, Tao Wang, Qian Wang, Chao Shen, and Yixiao Xu. 2021. An automated multi-tab website fingerprinting attack. *IEEE Transactions on Dependable and Secure Computing* 19, 6 (2021), 3656–3670.

[44] Xiyuan Zhao, Xinhao Deng, Qi Li, Yunpeng Liu, Zhuotao Liu, Kun Sun, and Ke Xu. 2024. Towards fine-grained webpage fingerprinting at scale. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 423–436.

Table 7: Parameter Settings

Module Part	Design Details	Value
Sequence Division	Channels	8
	Interval	20
	Signal Duration	8000
	Length alignment	truncate/pad to seq_len
Local Profiling	Number of blocks	4
	Number of filters	[32, 64, 128, 256]
	Kernel size	[7, 7, 7, 7]
	Pool (kernel/stride)	[(8/4), (8/4), (8/4), (8/4)]
	Dropout (per stage)	0.1
	Output dimension	256
Website Identification	Number of Transformer blocks	2
	Number of heads	8
	Dimension of head	32
	Embedding dimension	256
	Dimension of MLP	1024
	Positional embedding	learnable, length = max_len
	Output dimension	num_classes

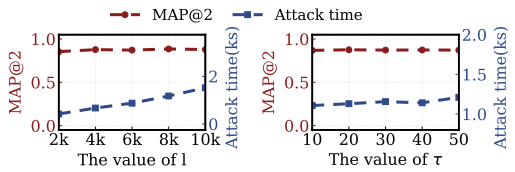


Figure 10: Parameter analysis of Forge. We select the interval τ and sequence length l.

A Parameter Settings

We choose the parameters with the best performance after tests. The settings of parameter values are shown in Table 7.

Parameter Analysis: We evaluate Forge 's sensitivity to its key parameters, the aggregation interval τ and sequence length l on $D_{\text{CW-2tab}}$ (Figure 10). The system is robust: varying τ from 10 ms to 50 ms keeps MAP@2 between 0.838 and 0.885; performance is similarly insensitive to l. This stability indicates effectiveness stems from the architecture rather than fine-tuning. By default, we set τ=20 and l=8000 to balance time and accuracy.

B Dataset Details

The experiments presented in this paper utilize the publicly available multi-tab WF datasets from the WFLib collection [11]. These datasets contain encrypted traffic traces from realistic multi-tab browsing sessions over Tor. The primary feature extracted and used from each trace is the packet direction sequence.

The WFLib collection is structured into three main categories: closed-world, open-world, and defended scenarios.

B.1 Closed-World Datasets

This dataset models a scenario where users only visit websites from a predefined monitored set of 100 top Alexa websites. Each traffic instance represents a browsing session with k concurrent tabs, where k is set from two to five. The k websites for each session are a specific combination randomly selected from the 100 monitored sites. For each k-tab setting, the dataset contains thousands of unique website combinations. To ensure data quality, each combination included in the final dataset has at least 10 recorded traffic instances.

B.2 Open-World Datasets

This dataset simulates a more realistic scenario where users also visit unmonitored websites. A k-tab open-world instance is composed of $k-1$ websites from the monitored set (the same 100 top sites) and one website from a large unmonitored pool (Alexa top 101 to 20,000). All websites from the unmonitored pool are assigned a single, common label. Due to the vast number of possible combinations, each unique open-world website combination was collected only once. The dataset ensures that each selected unmonitored website is used in only one instance.

B.3 Defense Datasets

This category is designed to evaluate WF resilience against countermeasures. Some defended traffic was collected under a two-tab, closed-world setting, while some was collected under a single-tab setting. Two distinct 2-tab defense datasets are provided, each corresponding to a specific defense mechanism applied to the traffic. Each of the defense datasets contains 20,000 2-tab traffic instances. The defenses are:

Front [16]: Front applies strong padding and timing randomization in early session phases, where classifiers extract the most discriminative features, and relaxes shaping thereafter. This degrades early-stage identifiability with low to moderate, front-loaded overhead. Deployment is moderately difficult, as effectiveness relies on accurate phase detection across heterogeneous sites, and is enhanced when combined with lightweight later-phase padding.

WTF-PAD [23]: WTF-PAD is an adaptive padding defense that targets burst-level features by probabilistically injecting dummy traffic during idle gaps and smoothing burst boundaries, reducing deterministic patterns while preserving responsiveness. Its overhead is typically low to moderate, with tens-of-percent bandwidth cost and modest latency, as padding is focused on idle periods. Deployment is moderately difficult: while it integrates cleanly with Tor, performance is sensitive to parameter choices, with conservative settings offering limited protection and aggressive padding degrading performance.

Additionally, this collection provides two single-page datasets, namely:

Walkie-Talkie [40]: Walkie-Talkie enforces half-duplex communication and aligns flows to precomputed templates, causing traces from different sites to collide under burst-based features. While this provides strong theoretical obfuscation, it incurs high latency and bandwidth overhead due to half-duplex alternation and template padding. Deployment is challenging given modern web dynamism, CDN variability, and content churn, which complicate stable template maintenance and degrade user experience.

TrafficSliver [10]: TrafficSliver confuses classifiers by slicing or replicating session portions across auxiliary paths or decoy flows, breaking the one-to-one mapping between traces and sites. Overhead is variable and often high, increasing with noise and duplication; latency can be mitigated if primary paths are prioritized, though multipath coordination may cause delays. Deployment is

Table 8: Datasets overview

Datasets	# of monitored websites	# of instances
Closed_2tab.npz	100	58,000
Closed_3tab.npz	100	58,000
Closed_4tab.npz	100	58,000
Closed_5tab.npz	100	58,000
Open_2tab.npz	100	64,000
Open_3tab.npz	100	64,000
Open_4tab.npz	100	64,000
Open_5tab.npz	100	64,000
WTF-PAD.npz	95	105,730
Front.npz	95	95,000
Walkie-Talkie.npz*	100	90,000
TrafficSliver.npz*	95	95,000

* Single-tab dataset.

challenging, requiring client/network support and careful policy design to prevent trivial de-noising, with infrastructure and operational complexity as main barriers.

Additionally, we simulated defense deployments on our $D_{CW-2tab}$ and constructed two datasets; the simulation procedures are detailed in the Appendix. The two defenses are, respectively, random noise injection and:

RegulaTor [19]: RegulaTor is a surge-shaped regularization defense that standardizes Tor download bursts and synchronizes uploads to suppress early-phase and burst-level identifiers. It starts downloads at a fixed, decaying rate, triggers new surges when queues exceed thresholds, and fills idle slots with dummy cells; uploads follow a fixed ratio with capped delay. Overhead is mainly bandwidth (tens-of-percent to <100%) with low latency, outperforming constant-rate schemes and comparable to adaptive padding. Deployment is straightforward as a pluggable transport or near-entry padding module, with parameters transferring well across datasets.

Other heavyweight defenses, such as Tamaraw [4] and BuFLO [3], can theoretically provide strong protection by enforcing constant-rate transmission that removes time- and frequency-domain features, but are impractical in real-world settings and thus not evaluated here.

B.4 Dynamic Datasets

Motivated by practical uncertainty—where the number of open tabs and the deployed WF defense may not be directly observable—we construct evaluation datasets by randomly sampling and mixing traffic instances under different conditions.

Dynamic multi-tab datasets. We randomly sample and mix 100,000 closed-world instances and 20,000 open-world instances from the multi-tab corpus. The tab-count distribution is fixed as follows: 40% with 2 tabs, 30% with 3 tabs, 20% with 4 tabs, and 10% with 5 tabs.

Dynamic defense datasets. For datasets with different monitoring settings cannot be mixed even if they share the same label dimensionality, to ensure label alignment, we only mix instances collected under defenses that share the same monitoring set. In our experiments, this criterion is met by WTF-PAD and Front. We therefore randomly sample and mix 30,000 instances in total, drawn equally from these two defenses (15,000 WTF-PAD and 15,000 Front).

C Single-tab Methods Adaptation

Referring to similar work [12], we adapt four single-label WF attack models—AWF, DF, Tik-Tok and RF—so they operate in a multi-label regime. We use binary cross-entropy(BCE) to supervise independent sigmoid outputs per class, enabling the models to learn multiple concurrent labels for each trace and thereby complete the transition from single-label to multi-label classification.

Let $\mathbf{x}$ denote the input and $\mathbf{y} \in \{0,1\}^K$ the multi-hot label vector over K sites/tabs. A model f_θ produces logits $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$ and posterior estimates via a sigmoid layer:

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}), \quad \sigma(z_k) = \frac{1}{1+e^{-z_k}}. \tag{7}$$

The multi-label training objective is the BCE loss:

$$\mathcal{L}_{\text{BCE}}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} \Big[y_k \log \hat{y}_k + (1-y_k) \log(1-\hat{y}_k) \Big] \tag{8}$$

.

Optionally, a class-imbalance prior with positive weights $w_k > 0$ yields:

$$\mathcal{L}_{\text{WBCE}} = -\sum_{k=1}^{K} \Big[w_k y_k \log \hat{y}_k + (1-y_k) \log(1-\hat{y}_k) \Big] \tag{9}$$

.

At inference time, predictions are made independently for each label as follows:

$$\hat{y}_k = \mathbb{I}\big[\hat{y}_k \geq \tau_k\big], \quad \tau_k \in (0,1) \tag{10}$$

, where thresholds $\{\tau_k\}$ can be tuned for desired precision–recall trade-offs.

Embedding-based approaches, while powerful, necessitate architectural surgery to construct a joint feature–label manifold . Such restructuring risks altering the core inductive biases of WF attacks and often increases inference latency—undesirable in our setting. Hence, we adopt the loss-centric route: for AWF, DF, RF and Tik-Tok, we retain the original backbones and simply (i) append a sigmoid output layer and (ii) replace the single-label loss with BCE as above. This minimally invasive modification preserves model design while enabling multi-label WF operation .

D Supplementary Data

For visual clarity, we display only five baselines in the figures. DF [34] and BAPM [18] are omitted due to their earlier publication dates. AWF [31] is not shown in the noise experiments because it is nearly nonfunctional. Here we provide their results across all experiments, as shown in the Tables 9, 10, 11, and 12.

Table 9: Performance of DF and BAPM on closed world on different tab settings.

	2		3		4		5	
	AUC	MAP@2	AUC	MAP@3	AUC	MAP@4	AUC	MAP@5
DF	0.9439	0.7016	0.8570	0.5634	0.8228	0.4997	0.7066	0.3042
BAPM	0.9348	0.6210	0.8703	0.4829	0.8423	0.4539	0.7973	0.3882

Table 10: AUC under different Tab settings for WF attacks.

Tab	2	3	4	5
WF Attack	AUC			
DF	0.9411	0.8604	0.8244	0.7790
BAPM	0.9320	0.8679	0.8367	0.7978

Table 11: Performance of DF and BAPM under Walkie-Talkie and Trafficsliver defenses.

	Walkie-Talkie		Trafficsliver	
	AUC	Pre	AUC	Pre
DF	0.7411	0.4874	0.7898	0.5841
BAPM	0.7117	0.4291	0.7307	0.4671

Table 12: AUC under different SNR levels for various WF attacks.

SNR Level	0	0.1	0.5	1	2	5	10
WF Attack	AUC						
AWF	0.7367	0.7253	0.7248	0.7285	0.7203	0.7072	0.6765
DF	0.9437	0.9387	0.9308	0.9168	0.8897	0.8680	0.8293
BAPM	0.9348	0.9249	0.9191	0.9126	0.8967	0.8541	0.8195

E Defense Simulation

Random defense: We simulate random defense by injecting dummy packets into $D_{\text{CW-2tab}}$ at a random rate below 20%, yielding 50,000 instances. RegularTor [19]: Using the simulator[1] with minimal modifications for our traces, we generate two variants, $D_{\text{RegularTor-Light}}$ and $D_{\text{RegularTor-Heavy}}$, keeping the instance count unchanged. Parameter settings are listed in Table 13.

Table 13: RegularTor parameter settings.

Config	orig rate	depreciation rate	max padding budget	burst threshold	upload ratio	delay cap [s]
Light	260.0	0.86	2080	3.75	3.95	1.77
Heavy	277.0	0.94	3550	3.55	3.95	1.77

F Noise Simulation

We implement a configurable noise-injection pipeline to emulate real-world network artifacts—burst packet loss, temporal jitter, and local reordering—applied sequentially to each flow. Using $\boldsymbol{D}_{\text{CW-2tab}}$ as the base, we generate six datasets with varying noise strengths ($s \in \{0.1, 0.5, 1, 2, 5, 10\}$).

Data representation and preprocessing. Each flow is a 1D sequence $X \in \mathbb{R}^L$. We split and merge via

$$t = |X|, \quad d = \text{sign}(X) \in \{-1, 0, +1\}, \qquad X^{\text{noisy}} = t' \odot d'.$$

After noise injection, sequences are compressed (if packets are dropped) and padded to the target length L; a binary mask mask $\in \{0, 1\}^L$ marks valid positions. To preserve causality, we enforce non-decreasing timestamps on modified segments using

$$t \leftarrow \max\text{-accumulate}(t) \quad \text{or} \quad t \leftarrow \max\text{-accumulate}(t + j).$$

[1] https://github.com/jkhollandjr/RegulaTor/

Table 14: Noise schedules: SNR-to-application-ratio mapping.

Nominal SNR level s	Loss application ratio α_{loss}	Jitter application ratio α_{jit}	Reorder application ratio α_{reorder}
0.1	0.1	0.1	0.01
0.5	0.5	0.5	0.05
1	1	1	0.1
2	2	2	0.2
5	5	5	0.5
10	10	10	1

Noise operators. Burst loss: Burst loss model. Consider a two-state Markov chain with states Good and Bad and transition probabilities

$$P(\text{Good} \to \text{Bad}) = p_{gb}, \qquad P(\text{Bad} \to \text{Good}) = p_{bg}.$$

In the Bad state, a packet is dropped with probability p_{drop}. Dropped packets are removed from the sequence, and timestamps are compressed accordingly.
Temporal jitter (AR(1) + optional heavy tail): $\epsilon_k \sim \mathcal{N}(0, \sigma^2)$, $j_k = \rho j_{k-1} + \epsilon_k$; with probability p_{tail}, add a log-normal impulse of scale $s \in [s_{\min}, s_{\max}]$. Apply monotonic repair after adding j.
Local reordering: within window size w, swap adjacent elements with probability p_{swap}; repair monotonicity locally.

All operators support range-based selection and an application ratio α controlling the fraction of positions affected within selected ranges.

Configuration. Master: seed, target length L, compress-after-drop flag, padding values (time, direction), return-mask flag.
Loss (GE): enabled, p_{gb}, p_{bg}, p_{drop}, ranges, application ratio α_{loss}.
Jitter (AR(1)+tail): enabled, σ (ms), ρ, p_{tail}, tail scale $[s_{\min}, s_{\max}]$ (ms), ranges, application ratio α_{jit}.
Reorder (local): enabled, p_{swap}, window w, ranges, application ratio α_{reorder}.

Default "shape" parameters used across all schedules: $p_{gb} = 0.003$, $p_{bg} = 0.25$, $p_{\text{drop}} = 0.2$; $\sigma = 2\,\text{ms}$, $\rho = 0.6$, $p_{\text{tail}} = 0$, $s \in [120\,\text{ms}, 500\,\text{ms}]$; $p_{\text{swap}} = 10^{-4}$, $w = 5$; ranges set to the full segment $[0, L)$.

SNR-to-ratio mapping. Let $s \in \{0.1, 0.5, 1, 2, 5, 10\}$ denote the nominal SNR level. We set $\alpha_{\text{loss}} = s$, $\alpha_{\text{jit}} = s$, $\alpha_{\text{reorder}} = 0.1s$. Keeping the shape parameters fixed and varying only α yields six datasets of increasing severity when applied to $D_{\text{CW-2tab}}$. The detailed parameter configurations for these datasets are summarized in Tables 14.

Execution and reproducibility. We fix the random seed (e.g., 1234) and apply operators in the order loss → jitter → reordering, repairing monotonicity after each step. The full dataset is processed in mini-batches for memory efficiency. For each s in the set above, we inject noise into all samples of $D_{\text{CW-2tab}}$, obtaining six companion datasets for robustness evaluation.

Practical notes. Larger α increases activation density within the selected ranges; scale α linearly if your sampling rate or flow density differs. Operator overlap is allowed; ranges can be customized per operator to model staged congestion and recovery. Output masks should be used during training to ignore padded positions in losses/metrics.